INTRODUCTION TO PYTHON
UNIT 1: PYTHON BASICS, VARIABLES, EXPRESSIONS & STATEMENTS
Curriculum: Lovely Professional University | School of Electronics & Communication Engineering
Reference: Python Programming: Using Problem Solving Approach by Reema Thareja

>> 1.1 Python Programming Paradigm and Execution Model

Python is a modern, high-level, interpreted, dynamically-typed programming language created by Guido van Rossum:
* Key Features: Clean readable syntax (PEP 8 standard), automatic memory management via garbage collection, dynamically typed (variables bind to objects, not types), platform independent (bytecode compiled to .pyc and executed on CPython Virtual Machine).
* Execution Modes: Interactive Shell (REPL - Read-Eval-Print Loop) for rapid prototyping vs Script Mode (.py files) for structured engineering software.
* Documentation & Comments: Single-line comments starting with #; multi-line documentation strings (Docstrings) enclosed in triple quotes (""" or ''') accessible via help() and __doc__ attribute.

>> 1.2 Data Types, Dynamic Typing and Variable Memory Model

* Primitive Built-in Types: int (arbitrary precision integer), float (64-bit IEEE 754 double precision), bool (True or False, subclass of int), complex (real + imag*1j), str (immutable Unicode sequence).
* Dynamic Typing: Variable names are merely references (pointers) to objects in heap memory. Reassigning a variable re-binds the pointer to a new object.
* Inspection Utilities: type(x) returns object data type; id(x) returns unique memory address identity; isinstance(x, type) verifies inheritance compatibility.
* Type Conversion (Casting): Implicit casting (e.g., int + float automatically promotes to float) vs Explicit casting using constructor functions (int(), float(), str(), bool(), complex()).

>> 1.3 Operators, Expressions and Precedence Hierarchy

* 1. Arithmetic Operators: Addition (+), Subtraction (-), Multiplication (*), True Division (/ yields float), Floor Division (// truncates towards negative infinity), Modulus (% remainder), Exponentiation (**).
* 2. Relational / Comparison Operators: Equal (==), Not Equal (!=), Greater Than (>), Less Than (<), Greater/Equal (>=), Less/Equal (<=). Can be chained: 0 <= x < 10.
* 3. Logical Operators: and, or, not. Utilize short-circuit evaluation (x and y returns x if x is falsy, otherwise y).
* 4. Bitwise Operators: Operates directly on binary bit representations: Bitwise AND

(&), Bitwise OR (|), Bitwise XOR (^), Bitwise Inversion (~x = -(x + 1)), Left Shift (<< equivalent to multiplying by 2^n), Right Shift (>> equivalent to integer dividing by 2^n). Essential for embedded microcontroller register programming.
* 5. Identity & Membership: is, is not (compares memory addresses id(a) == id(b)) vs in, not in (tests membership in sequences).
* Precedence Hierarchy: Parentheses () > Exponentiation ** > Unary +,-,~ > Multiplication/Division/Modulus *,/,//,% > Addition/Subtraction +,- > Bitwise Shifts <<,>> > Bitwise & > Bitwise ^ > Bitwise | > Comparisons ==,!=,<=,>= > Logical not > Logical and > Logical or.

>> 1.4 Standard Input/Output and String Formatting

* Standard Input: input(prompt) reads a line from console as a string. Must be explicitly wrapped in int() or float() for numerical computations.
* Standard Output: print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False).
* Modern String Formatting: f-strings (Formatted String Literals, introduced in Python 3.6): f"Value: {var:.2f}" evaluates expressions inline with format specifiers.

[PROGRAM 1: AC Electrical Power and Complex Impedance Calculator]

Problem Context: Calculate real power, reactive power, apparent power, and power factor for an alternating current circuit given voltage V_rms, current I_rms, and phase angle theta.
Python Source Code:

```
import math

# Input circuit parameters
v_rms = float(input('Enter RMS Voltage (Volts): '))
i_rms = float(input('Enter RMS Current (Amperes): '))
theta_deg = float(input('Enter Phase Angle (degrees, + for lag, - for lead): '))

# Convert angle to radians for trigonometric functions
theta_rad = math.radians(theta_deg)

# Electrical Power Computations
p_active = v_rms * i_rms * math.cos(theta_rad) # Watts
q_reactive = v_rms * i_rms * math.sin(theta_rad) # VAR
s_apparent = v_rms * i_rms # VA
power_factor = math.cos(theta_rad)
pf_nature = 'Lagging (Inductive)' if theta_deg > 0 else ('Leading (Capacitive)' if theta_deg < 0 else 'Unity')

# Complex Impedance Calculation: Z = V / I
```

```
z_magnitude = v_rms / i_rms
z_complex = complex(z_magnitude * math.cos(theta_rad), z_magnitude *
math.sin(theta_rad))

print('\n--- ELECTRICAL POWER & IMPEDANCE REPORT ---')
print(f'Apparent Power (S) : {s_apparent:10.2f} VA')
print(f'Active Power (P) : {p_active:10.2f} Watts')
print(f'Reactive Power (Q) : {q_reactive:10.2f} VAR')
print(f'Power Factor (PF) : {power_factor:10.4f} ({pf_nature})')
print(f'Complex Impedance : {z_complex.real:.2f} + {z_complex.imag:.2f}j Ohms')
```

[PROGRAM 2: Quadratic Equation Solver with Complex Conjugate Roots]

Problem Context: Solve the general quadratic equation a*x^2 + b*x + c = 0, handling real distinct roots, repeated roots, and complex conjugate roots.

Python Source Code:

```
import cmath

def solve_quadratic(a, b, c):
if a == 0:
return 'Linear equation: root = ' + str(-c / b) if b != 0 else 'Invalid equation'

# Calculate discriminant: D = b^2 - 4ac
discriminant = b**2 - 4 * a * c

if discriminant > 0:
r1 = (-b + discriminant**0.5) / (2 * a)
r2 = (-b - discriminant**0.5) / (2 * a)
return f'Real & Distinct Roots: x1 = {r1:.4f}, x2 = {r2:.4f}'
elif discriminant == 0:
r = -b / (2 * a)
return f'Real & Repeated Roots: x1 = x2 = {r:.4f}'
else:
# Complex roots using cmath
r1 = (-b + cmath.sqrt(discriminant)) / (2 * a)
r2 = (-b - cmath.sqrt(discriminant)) / (2 * a)
return f'Complex Conjugate Roots: x1 = {r1.real:.4f} + {r1.imag:.4f}j, x2 =
{r2.real:.4f} + {r2.imag:.4f}j'

# Demonstration executions
print('Case 1 (Real):', solve_quadratic(1, -5, 6))
print('Case 2 (Repeated):', solve_quadratic(1, -4, 4))
print('Case 3 (Complex):', solve_quadratic(2, 4, 10))
```

[PROGRAM 3: Embedded Microcontroller Register Bit Manipulation]

Problem Context: Demonstrate bitwise masking, bit-setting, bit-clearing, and bit-toggling operations on an 8-bit microcontroller GPIO Port register.

Python Source Code:

```
# Initial 8-bit GPIO Data Direction Register (DDR): 0b00110100 (Decimal 52)
port_reg = 0b00110100

def print_binary(label, val):
print(f'{label:<25} : 0b{val:08b} (Decimal {val})')

print_binary('Initial Register', port_reg)

# 1. SET bit 1 (Make pin 1 an output): port_reg |= (1 << 1)
port_reg |= (1 << 1)
print_binary('After SET Bit 1', port_reg)

# 2. CLEAR bit 5: port_reg &= ~(1 << 5)
port_reg &= ~(1 << 5)
print_binary('After CLEAR Bit 5', port_reg)

# 3. TOGGLE bit 2: port_reg ^= (1 << 2)
port_reg ^= (1 << 2)
print_binary('After TOGGLE Bit 2', port_reg)

# 4. READ / TEST bit 4:
is_bit4_set = bool(port_reg & (1 << 4))
print(f'Is Bit 4 Set? : {is_bit4_set}')
```

[PROGRAM 4: Multi-Scale Temperature Converter with Engineering Table]

Problem Context: Convert input temperatures between Celsius, Fahrenheit, and Kelvin, and output a tabulated thermodynamic calibration chart.

Python Source Code:

```
temps_celsius = [-40.0, 0.0, 25.0, 37.0, 100.0, 200.0]

print(f'{'Celsius (C)':<15} | {'Fahrenheit (F)':<15} | {'Kelvin (K)':<15} | {'State
Description'}')
print('-' * 65)

for c in temps_celsius:
f = (c * 9/5) + 32
```

```
k = c + 273.15
desc = 'Freezing Point of Water' if c == 0 else ('Boiling Point' if c == 100 else ('Room
Temp' if c == 25 else ('Body Temp' if c == 37 else 'Industrial Temp')))
print(f'{c:<15.2f} | {f:<15.2f} | {k:<15.2f} | {desc}')
```